

DATA AUDIT · CLEANING PIPELINE · SEMANTIC MODEL

NCR Ride Booking Data Blueprint

A full profile of the 150,000-row booking extract, the Power Query pipeline that turns it into a trustworthy table, and the star schema the five dashboard views should sit on.

Source ncr_ride_bookings.csv · 24.1 MB

Model NCR Ride Booking Executive Analytics Dashboard.pbip

Profiled all 150,000 rows, not a sample

150,000	21	365	0
ROWS	COLUMNS	DAYS, NO GAPS	DUPLICATE RECORDS
1,224	100%		
COLLIDING BOOKING IDS	IDS CARRY QUOTE MARKS		

STEP 01 Data quality & structure audit

Every column profiled against the full 150,000 rows. The headline is unusual and worth stating up front: **this dataset has almost no dirty data** — but it has three real structural defects, and 32–94% of its cells are legitimately empty by design.

1.1 Column inventory

#	COLUMN	TYPE IN MODEL	NULLS	NULL %	DISTINCT
1	Date	Date	0	0.00%	365
2	Time	Time	0	0.00%	42

2	NAME	TYPE	0	0.00%	02,
3	Booking ID	Text	0	0.00%	148,
4	Booking Status	Text	0	0.00%	
5	Customer ID	Text	0	0.00%	148,
6	Vehicle Type	Text	0	0.00%	
7	Pickup Location	Text	0	0.00%	
8	Drop Location	Text	0	0.00%	
9	Avg VTAT	Decimal	10,500	7.00%	
10	Avg CTAT	Decimal	48,000	32.00%	
11	Cancelled Rides by Customer	Whole	139,500	93.00%	
12	Reason for cancelling by Customer	Text	139,500	93.00%	
13	Cancelled Rides by Driver	Whole	123,000	82.00%	
14	Driver Cancellation Reason	Text	123,000	82.00%	
15	Incomplete Rides	Whole	141,000	94.00%	
16	Incomplete Rides Reason	Text	141,000	94.00%	
17	Booking Value	Whole	48,000	32.00%	2,
18	Ride Distance	Decimal	48,000	32.00%	4,
19	Driver Ratings	Decimal	57,000	38.00%	
20	Customer Rating	Decimal	57,000	38.00%	
21	Payment Method	Text	48,000	32.00%	

Distinct counts exclude nulls. Types shown are what Power BI has currently inferred.

1.2 The null map — the single most important table here

Before deciding what to do about missing values, you have to know whether

they are *failures* or *facts*. Cross-tabulating every measure against Booking Status gives an answer with no ambiguity in it at all:

BOOKING STATUS	ROWS	AVG VTAT	AVG CTAT	BOOKING VALUE	RIDE DISTANC
Completed	93,000	93,000	93,000	93,000	93,00
Incomplete	9,000	9,000	9,000	9,000	9,00
Cancelled by Driver	27,000	27,000	0	0	
Cancelled by Customer	10,500	10,500	0	0	
No Driver Found	10,500	0	0	0	

Cells show the count of *populated* values. Every figure is either the full row count or exactly zero — there is not one partial cell in the matrix.

Read down the columns and the operational story falls out cleanly:

- **No Driver Found** has no VTAT because no vehicle was ever allocated — there was nothing to time.
- **Both cancellation states** have VTAT but nothing after it: a driver was assigned and en route, then the trip died before pickup. No trip, no fare, no rating.
- **Incomplete** rides have fare, distance *and* payment method — the trip started and was billed — but no ratings, because neither side rates a ride that never ended. **DESIGN CONSEQUENCE** Incomplete rides are revenue-generating and must be included in revenue measures.

VERDICT ON MISSING VALUES

Zero of the 1,088,500 empty cells are data-quality problems. Every one is a structurally expected absence explained entirely by booking status. Nothing should be imputed, filled with zero, or dropped. Filling them would invent revenue that was never earned and ratings nobody ever gave.

1.3 Consistency & referential integrity

Nine cross-field rules were tested. Every one passes with a count of zero

exceptions:

INTEGRITY RULE	VIOLATIONS	RESULT
Customer-cancellation flag set on a row whose status is not <i>Cancelled by Customer</i>	0	PASS
Status is <i>Cancelled by Customer</i> but the flag is missing	0	PASS
Status is <i>Cancelled by Customer</i> but no cancellation reason	0	PASS
Driver-cancellation flag set on a row whose status is not <i>Cancelled by Driver</i>	0	PASS
Status is <i>Cancelled by Driver</i> but the flag is missing	0	PASS
Status is <i>Cancelled by Driver</i> but no cancellation reason	0	PASS
Incomplete flag set on a row whose status is not <i>Incomplete</i>	0	PASS
Status is <i>Incomplete</i> but the flag is missing	0	PASS
Status is <i>Incomplete</i> but no incompleteness reason	0	PASS

Because the three flag columns are perfectly determined by Booking Status and are constant at 1 wherever they appear, they carry no information the status column doesn't already hold. All three should be removed.

1.4 Uniqueness & duplicates

Whole-row duplicates

0 of 150,000. No de-duplication step is needed, and adding *Remove Duplicates* blindly would be a mistake.

Booking ID

148,767 distinct values for 150,000 rows. 1,224 IDs appear more than once (up to 3×), accounting for 1,233 surplus rows.

Customer ID

148,788 distinct customers.
147,582 of them booked exactly once; 1,200 booked twice; 6 booked three times

BOOKED THREE TIMES.

BOOKING ID IS NOT A PRIMARY KEY

The repeats are not duplicate records — they are genuinely different rides that were handed the same identifier. One example of CNR7908610 :

DATE	TIME	STATUS	CUSTOMER	VEHICLE	VALUE
2024-03-26	15:54:11	Completed	CID7755899	Uber XL	707
2024-09-03	19:38:29	Completed	CID5251969	Go Mini	456
2024-12-30	07:01:22	Completed	CID3266637	Bike	495

Different day, customer, vehicle and fare. These must all be kept. The fix is a generated surrogate key, not deletion.

1.5 ID formatting defect

Every row of both ID columns is wrapped in literal double-quote characters. This is not a CSV artefact you can ignore — it is *in the loaded model*. A DAX probe against the live semantic model returns "CNR1984811" , quotes included, as an actual character value.

```
-- raw bytes in the CSV
3/23/2024,12:29:38,"""CNR5884300""",No Driver Found,"""CID1982111""",eBike,...

-- what the model currently stores
Booking ID  = "CNR5884300"      ← the quote marks are part of the string
Customer ID = "CID1982111"
```

Root cause: your current query uses `QuoteStyle = QuoteStyle.None` , which tells Power Query to treat quote characters as ordinary text instead of as field delimiters. Underneath the quoting, both IDs are perfectly formed — CNR / CID plus exactly seven digits, with 0 pattern violations across all 300,000 values.

1.6 Value validity, ranges and outliers

COLUMN	MIN	P25	MEDIAN	P75	MAX	MEAN	IQR	OUTLIERS
Avg VTAT	2.0	5.3	8.3	11.3	20.0	8.46		0
Avg CTAT	10.0	21.6	28.8	36.8	45.0	29.15		0
Booking Value	50	234	414	689	4,277	508.30		3,435
Ride Distance	1.00	12.46	23.72	36.82	50.00	24.64		0
Driver Ratings	3.0	4.1	4.3	4.6	5.0	4.231		0
Customer Rating	3.0	4.2	4.5	4.8	5.0	4.405		0

No zeros and no negative values anywhere. Ratings are single-decimal, 21 discrete levels each.

On the 3,435 “outlier” booking values

They sit above the 1.5×IQR fence of ₹1,371 — 3.37% of monetised rides. That is a *fat right tail, not corruption*: surge pricing, airport runs and long inter-city hops legitimately produce fares up to ₹4,277. The one figure worth watching is implied fare rate: the median is a sensible ₹19/km, but the maximum reaches ₹3,148/km on a 1 km trip. That is a minimum-fare or heavy-surge artefact rather than an error. **Keep all rows.** Flag them with a measure if you want an executive footnote; never delete them.

Date and time validity

- 0 unparseable dates, 0 unparseable times, 0 future dates.
- 2024-01-01 to 2024-12-30 — 365 distinct days with no gaps. 355 to 462 bookings per day, mean 411.
- Source format is M/D/YYYY , US-ordered. Your model culture is en-IN . It has parsed correctly, but this is a locale trap on any refresh from a differently-configured machine and must be pinned explicitly.
- The hourly distribution is realistically bimodal — a trough of ~1,300

rides/hour at 02:00–04:00, peaks of 11,000–12,400 at 17:00–19:00 and a secondary morning peak at 09:00–10:00. Time-of-day is genuinely analysable.

Should Date and Time be combined?

Yes — as an addition, not a replacement. Create a Booking DateTime for exact sequencing and any future duration work, and derive a Ride Hour for time-of-day slicing. But *keep Date as a standalone date column*: it is what joins to the Date dimension, and a datetime with 62,910 distinct time values would make a useless relationship key and bloat the model.

1.7 Categorical value integrity

Checked for casing variants, stray whitespace and near-duplicate spellings across all eight categorical columns. **Result: nothing to standardise.** Zero leading/trailing spaces, zero casing collisions, closed and consistent domains throughout.

Booking Status 5 VALUES

Completed · Cancelled by Driver · No Driver Found · Cancelled by Customer · Incomplete

Vehicle Type 7 VALUES

Auto · Go Mini · Go Sedan · Bike · Premier Sedan · eBike · Uber XL

Payment Method 5 VALUES

UPI · Cash · Uber Wallet · Credit Card · Debit Card

Cancellation & incompleteness reasons

12 VALUES

5 customer · 4 driver · 3 incomplete

Locations are equally clean: the 176 pickup localities and 176 drop localities are the *same* set, and no row has pickup equal to drop.

1.8 Findings summary

F1

ID columns contain literal quote characters CRITICAL

All 150,000 rows of both Booking ID and Customer ID . Breaks joins, breaks slicer search, and shows quote marks to users in every “Top 5 Customers” visual. Caused by QuoteStyle.None in the source step.

F2 Booking ID cannot serve as a primary key CRITICAL

1,224 identifiers are reused across genuinely distinct rides. A surrogate Ride Key index must be generated; the natural key stays as a display attribute only.

F3 Three redundant flag columns REMOVE

Cancelled Rides by Customer , Cancelled Rides by Driver and Incomplete Rides are constant 1 and fully derivable from Booking Status . They add 3 columns and no information, and invite double-counting if someone drags them into a visual.

F4 Auto date/time is on and generating hidden tables FIX

The model currently carries DataTableTemplate_cc4684c5 and LocalDateTable_484aa8e1 plus an auto-relationship. Turn auto date/time off and use one governed Date dimension instead.

F5 Date parsing is not locale-pinned FIX

M/D/YYYY source read under an en-IN model. Correct today, fragile on refresh. Pin the culture in the type-change step.

F6 Nine column names are not report-ready RENAME

Acronyms (Avg VTAT , Avg CTAT), inconsistent singular/plural on the two rating columns, and a lowercase-verb name in Reason for cancelling by Customer . These names appear verbatim on every axis and tooltip.

F7 No imputation required anywhere CONFIRMED

All missing values are structurally expected. No filling, no dropping, no default zeros.

F8 No duplicate records, no invalid categories, no impossible numbers

CONFIRMED

Ranges, domains, dates and times all validate. The dataset is fundamentally sound.

STEP 02 **Power Query cleaning pipeline**

Fourteen transformations, in dependency order. The governing principle:
Power Query for anything that is a fixed property of the row, DAX for
anything that must respond to what the user has filtered.

2.1 Transformation register

#	TRANSFORMATION	WHERE	WHY IT IS REQUIRED
1	Switch to <code>QuoteStyle.Csv</code>	POWER QUERY	Root fix for F1. Tells the parser that quotes delimit fields rather than being content. Every downstream ID cleanup depends on this being right first.
2	Literal <code>"null"</code> text → real null	POWER QUERY	The CSV writes the four characters <code>n-u-l-l</code> , not an empty field. Without this replacement a text column keeps the string and a numeric conversion throws an error row.
3	Strip quotes from both ID columns	POWER QUERY	Belt and braces behind step 1, and it also handles any residual whitespace. Must happen before the index, so the key is built on final values.
4	Trim + clean all text columns	POWER QUERY	Defensive. The current extract is clean, but <code>Text.Clean</code> also strips non-printing control characters that silently break grouping on future refreshes.
5	Drop the three flag columns	POWER QUERY	F3. Removing at source is better than hiding: hidden columns still consume memory and still appear in the field list for anyone with edit rights.
6	Date conversion, culture-pinned	POWER QUERY	F5. <code>type date</code> with an explicit <code>"en-US"</code> culture argument forces M/D/YYYY interpretation regardless of the refreshing machine's regional settings.
7	Time conversion	POWER QUERY	<code>type time</code> , not <code>datetime</code> . A time-only column keeps the column compressible and stops Power BI attaching an auto date hierarchy to it.
8	Numeric type assignment	POWER QUERY	Fixing types in the query rather than the model keeps the definition in one place and lets the VertiPaq engine choose the right compression at load.

9	Create Booking DateTime	POWER QUERY	Exact event ordering, deterministic surrogate keys, and a foundation for any future gap or duration analysis. Additive — Ride Date survives as the relationship key.
10	Derive Ride Hour + Time of Day	POWER QUERY	The demand curve is strongly bimodal, so hour-of-day is a real analytical axis. Both are fixed row properties, computed once at refresh instead of on every visual render.
11	Derive Distance Band	POWER QUERY	Directly serves the <i>Ride Distance Distribution</i> histogram. A static bucket with fixed boundaries belongs in the table; only a user-adjustable bin size would justify a DAX parameter instead.
12	Generate Ride Key surrogate index	POWER QUERY	F2. Sort by Booking DateTime then Booking ID so the key is stable across refreshes, then add a 1-based index. This is the real primary key.
13	Rename nine columns	POWER QUERY	F6. These strings become axis titles, legend headers and tooltip labels. <i>Avg Pickup Wait (min)</i> needs no glossary; <i>Avg VTAT</i> does.
14	Reorder columns	POWER QUERY	Cosmetic but real: key → time → who → what → money → quality → exceptions makes the field list navigable when you are building 11 visuals.
—	Validate ratings / value / distance	DAX	Validation is a <i>report</i> , not a transformation. Build them as measures on a hidden QA page so a bad refresh surfaces visibly instead of silently dropping rows.
—	Fill or replace nulls	DO NOTHING	F7. Every null is structurally expected. Filling is data fabrication.

2.2 The null decision, stated explicitly

A True data-quality problems

Count: zero. No unexpected blanks, no partial records, no orphan values, no type failures. There is nothing in this dataset for null handling to repair.

B Structurally expected absences

Count: all 1,088,500. Each traces to a booking status where the value could not exist. Preserve every one as a genuine null.

WHY NOT REPLACE NULLS WITH “NOT APPLICABLE”?

It's tempting, because a blank category shows up as an empty slice in the cancellation pie charts. But a placeholder is worse than a blank: it enters the value domain, so it appears in slicers and legends, and every visual then needs a filter to exclude it — the same filter you would have written anyway, plus a fake category you now have to explain.

Do this instead. Filter the visual, not the data. On *Cancelled Rides Reasons (Customer)*, set a visual-level filter `Booking Status = "Cancelled by Customer"`. On *Revenue by Payment Method*, filter to `Payment Method is not blank`. The chart then means exactly what its title says.

2.3 On “standardising” the categoricals

Steps 6–9 of your brief ask for value standardisation on Vehicle Type, Booking Status, Payment Method and the reason texts. The audit found **nothing to standardise** — no casing variants, no whitespace, no misspellings. The honest answer is to keep only the defensive `Text.Trim / Text.Clean` pass (step 4) and *not* add a value-replacement step that changes nothing today and quietly hides drift tomorrow.

Two things are worth doing instead, and both belong in the **dimension tables**, not the fact:

- `Uber XL` and `eBike` are correct product names — don't reformat them. Add a **Vehicle Category** attribute (`Two-Wheeler / Auto / Hatchback / Sedan / SUV`) so the Vehicle Type view can roll up.
- Add an explicit **sort order** to Booking Status and Payment Method. Without it, every visual falls back to alphabetical and puts *Cancelled by Customer* ahead of *Completed*.

2.4 Complete M script

Open Home → Transform data → Advanced Editor on the `ncr_ride_bookings` query and replace its contents entirely:

```
let
    // 1 - SOURCE: QuoteStyle.Csv is the fix for the quoted IDs
    Source = Csv.Document(
```

```

File.Contents("C:\Users\MYPC\OneDrive\Documents\BIA\Power BI\August Assignm
[Delimiter = ",", Columns = 21, Encoding = 65001, QuoteStyle = QuoteStyle.C
),
Headers = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),

// 2 – the literal text "null" becomes a real null, in every column
RealNulls = Table.ReplaceValue(
    Headers, "null", null, Replacer.ReplaceValue, Table.ColumnNames(Headers)
),

// 3 – strip quote characters and whitespace from the identifiers
CleanIds = Table.TransformColumns(RealNulls, {
    {"Booking ID", each Text.Trim(Text.Remove(_, {""}))}, type text},
    {"Customer ID", each Text.Trim(Text.Remove(_, {""}))}, type text}
}),

// 4 – defensive trim + control-character clean on every text column
TextCols = {
    "Booking Status", "Vehicle Type", "Pickup Location", "Drop Location",
    "Reason for cancelling by Customer", "Driver Cancellation Reason",
    "Incomplete Rides Reason", "Payment Method"
},
Trimmed = Table.TransformColumns(
    CleanIds,
    List.Transform(TextCols, each {
        _, (v) => if v = null then null else Text.Trim(Text.Clean(v)), type tex
    })
),

// 5 – drop the three redundant flag columns (always 1, always implied by statu
Dropped = Table.RemoveColumns(Trimmed, {
    "Cancelled Rides by Customer", "Cancelled Rides by Driver", "Incomplete Rid
}),

// 6,7,8 – types. The "en-US" culture pins M/D/YYYY parsing.
Typed = Table.TransformColumnTypes(Dropped, {
    {"Date", type date}, {"Time", type time},
    {"Booking ID", type text}, {"Booking Status", type text},
    {"Customer ID", type text}, {"Vehicle Type", type text},
    {"Pickup Location", type text}, {"Drop Location", type text},
    {"Avg VTAT", type number}, {"Avg CTAT", type number},
    {"Reason for cancelling by Customer", type text},
    {"Driver Cancellation Reason", type text},
    {"Incomplete Rides Reason", type text},
    {"Booking Value", Int64.Type}, {"Ride Distance", type number},
    {"Driver Ratings", type number}, {"Customer Rating", type number},
    {"Payment Method", type text}
}, "en-US"),

// 9 – combined timestamp (additive; Date survives as the join key)
WithDateTime = Table.AddColumn(Typed, "Booking DateTime",
    each [Date] & [Time], type datetime)

```

```

each [date] & [time], type datetime),

// 10 - time-of-day attributes
WithHour = Table.AddColumn(WithDateTime, "Ride Hour",
    each Time.Hour([Time]), Int64.Type),
WithDaypart = Table.AddColumn(WithHour, "Time of Day", each
    if [Ride Hour] < 5 then "Late Night (00-04)"
    else if [Ride Hour] < 12 then "Morning (05-11)"
    else if [Ride Hour] < 17 then "Afternoon (12-16)"
    else if [Ride Hour] < 21 then "Evening (17-20)"
    else "Night (21-23)", type text),

// 11 - distance buckets for the histogram view (null stays null)
WithBand = Table.AddColumn(WithDaypart, "Distance Band", each
    if [Ride Distance] = null then null
    else if [Ride Distance] ≤ 5 then "00-05 km"
    else if [Ride Distance] ≤ 10 then "05-10 km"
    else if [Ride Distance] ≤ 20 then "10-20 km"
    else if [Ride Distance] ≤ 30 then "20-30 km"
    else if [Ride Distance] ≤ 40 then "30-40 km"
    else "40-50 km", type text),

// 12 - deterministic surrogate key (Booking ID is NOT unique)
SortedForKey = Table.Sort(WithBand, {
    {"Booking DateTime", Order.Ascending}, {"Booking ID", Order.Ascending}
}),
Keyed = Table.AddIndexColumn(SortedForKey, "Ride Key", 1, 1, Int64.Type),

// 13 - business-readable names
Renamed = Table.RenameColumns(Keyed, {
    {"Date", "Ride Date"},
    {"Time", "Ride Time"},
    {"Avg VTAT", "Avg Pickup Wait (min)"},
    {"Avg CTAT", "Avg Trip Duration (min)"},
    {"Booking Value", "Booking Value (INR)"},
    {"Ride Distance", "Ride Distance (km)"},
    {"Driver Ratings", "Driver Rating"},
    {"Reason for cancelling by Customer", "Customer Cancellation Reason"},
    {"Incomplete Rides Reason", "Incomplete Ride Reason"}
}),

// 14 - readable column order
Final = Table.ReorderColumns(Renamed, {
    "Ride Key", "Booking ID", "Customer ID",
    "Ride Date", "Ride Time", "Booking DateTime", "Ride Hour", "Time of Day",
    "Booking Status", "Vehicle Type", "Pickup Location", "Drop Location",
    "Avg Pickup Wait (min)", "Avg Trip Duration (min)",
    "Booking Value (INR)", "Payment Method",
    "Ride Distance (km)", "Distance Band",
    "Driver Rating", "Customer Rating",
    "Customer Cancellation Reason", "Driver Cancellation Reason",
    "Incomplete Ride Reason"
})

```

```
    })  
    in  
    Final
```

TWO SETTINGS TO CHANGE BEFORE YOU REFRESH

- 1. Turn off auto date/time. *File* → *Options and settings* → *Options* → *Current File* → *Data Load*, uncheck *Auto date/time*. This deletes `DateTableTemplate_cc4684c5` and `LocalDateTable_484aa8e1` and the auto relationship they carry.
- 2. Rename the query from `ncr_ride_bookings` to `Fact_Rides` — right-click in the Queries pane → *Rename*. Every measure and visual you write from here on references it by name.

2.5 Final table structure after cleaning

Fact_Rides — 150,000 rows × 23 columns (21 in, 3 dropped, 5 derived).

#	COLUMN	TYPE	ROLE	ORIGIN
1	Ride Key	Whole number	Primary key, hidden	DERIVED
2	Booking ID	Text	Natural key, display only	Cleaned
3	Customer ID	Text	Degenerate dimension	Cleaned
4	Ride Date	Date	Foreign key → Dim_Date	Renamed
5	Ride Time	Time	Attribute	Renamed
6	Booking DateTime	Date/time	Ordering, hidden	DERIVED
7	Ride Hour	Whole number	Attribute	DERIVED
8	Time of Day	Text	Attribute	DERIVED
9	Booking Status	Text	Foreign key → Dim_Booking_Status	Unchanged
10	Vehicle Type	Text	Foreign key → Dim_Vehicle	Unchanged
11	Pickup Location	Text	Attribute	Unchanged

		Text	Attribute	Unchanged
12	Drop Location	Text	Attribute	Unchanged
13	Avg Pickup Wait (min)	Decimal	Measure column	Renamed
14	Avg Trip Duration (min)	Decimal	Measure column	Renamed
15	Booking Value (INR)	Whole number	Measure column	Renamed
16	Payment Method	Text	Foreign key → Dim_Payment_Method	Unchanged
17	Ride Distance (km)	Decimal	Measure column	Renamed
18	Distance Band	Text	Attribute	DERIVED
19	Driver Rating	Decimal	Measure column	Renamed
20	Customer Rating	Decimal	Measure column	Unchanged
21	Customer Cancellation Reason	Text	Attribute	Renamed
22	Driver Cancellation Reason	Text	Attribute	Unchanged
23	Incomplete Ride Reason	Text	Attribute	Renamed

Set *Summarization: Don't summarize* on Ride Hour, and hide Ride Key and Booking DateTime from report view.

STEP 03 Semantic model design

One fact table, four dimensions, four single-direction relationships. Deliberately lean — a dimension earns its place only when it carries attributes the fact table cannot.

3.1 Single flat table, or star schema?

Star schema. The dataset arrives as one flat file, and it would technically work as one — but three of your five dashboard views need something the flat table cannot give:

- **Ride Volume Over Time** needs a continuous date axis and time intelligence.

Without a Date dimension there is no way to show a week with zero rides,

and no working `DATEADD / SAMEPERIODLASTYEAR` .

- **Booking Status Breakdown** needs a sensible slice order and a Successful/Cancelled/Unfulfilled rollup. Neither exists as a column in the fact.
- **Top 5 Vehicle Types** benefits from a category rollup so Sedan variants can be compared as a group.

What a star schema buys you here is not compression — the fact table is only 150,000 rows and would perform fine flat. It buys a **place to put business logic**: sort orders, groupings and labels that belong to the entity, not to the transaction.

3.2 Which dimensions to build, and which to refuse

CANDIDATE	ROWS	DECISION	REASONING
Dim_Date	366	BUILD	Non-negotiable. Time intelligence, continuous axis, month/weekday sorting, weekend flags.
Dim_Booking_Status	5	BUILD	Carries <i>Status Group</i> and <i>Is Revenue Generating</i> — real logic used across the Overall Performance and Revenue views. Also fixes slice ordering.
Dim_Vehicle	7	BUILD	Carries <i>Vehicle Category</i> and a display order. Cheap, and the natural home for capacity or base-fare attributes later.
Dim_Payment_Method	5	BUILD	Marginal but positive: gives the stacked bar a stable series order and a Cash / Card / Digital rollup.
Dim_Customer	148,788	REFUSE	The important call. 148,788 distinct customers across 150,000 rides — 147,582 of them booked exactly once. A dimension here would be nearly the same size as the fact, hold no attributes beyond the ID itself, and slow every query that crossed it. Keep <code>Customer ID</code> in the fact as a <i>degenerate dimension</i> ; <i>Top 5 Customers by Booking Value</i> works perfectly off it.
Dim_Location	176	DEFER	Would need role-playing (pickup and drop both point at it). which means two physical

			tables or an inactive relationship plus USERELATIONSHIP . None of your five views uses geography. Build it when you add a route or corridor analysis, not before.
Dim_Cancellation_Reason	12	REFUSE	The reasons are role-split — customer reasons and driver reasons are disjoint sets that never mix. A shared dimension would need two relationships to serve one purpose. Leave both as fact attributes.

3.3 Table specifications

Fact_Rides FACT · 150,000 ROWS · GRAIN: ONE BOOKING

Purpose — one row per booking attempt, whatever its outcome. Holds every additive and semi-additive measure column plus the four foreign keys.

Primary key — Ride Key (surrogate). Not Booking ID , which collides on 1,224 values.

Columns — the 23 listed in section 2.5.

Dim_Date 366 ROWS Purpose — the calendar spine for all time analysis. PK Date Columns Date, Year, Quarter Number, Quarter, Year Quarter, Month Number, Month, Month Short, Year Month, Month Year, Month Year Sort, Week Number, Year Week, Week Start Date, Day, Day of Week Number, Day of Week, Day of Week Short, Day Type, Is Weekend, Date Key	Dim_Booking_Status 5 ROWS Purpose — outcome taxonomy and its display order. PK Booking Status Columns Booking Status, Status Group, Is Revenue Generating, Status Sort
Dim_Vehicle 7 ROWS Purpose — product catalogue and its rollup.	Dim_Payment_Method 5 ROWS Purpose — payment channel catalogue and its rollup.

PK Vehicle Type
Columns Vehicle Type, Vehicle Category, Vehicle Sort

PK Payment Method
Columns Payment Method, Payment Category, Payment Sort

3.4 Relationship matrix

FROM (ONE)	TO (MANY)	CARDINALITY	CROSS
Dim_Date[Date]	Fact_Rides[Ride Date]	1 → *	Singl
Dim_Booking_Status[Booking Status]	Fact_Rides[Booking Status]	1 → *	Singl
Dim_Vehicle[Vehicle Type]	Fact_Rides[Vehicle Type]	1 → *	Singl
Dim_Payment_Method[Payment Method]	Fact_Rides[Payment Method]	1 → *	Singl

THE ONE RELATIONSHIP THAT NEEDS CARE

The 48,000 fact rows with no payment method have nowhere to land in the dimension, so Power BI adds an implicit blank member and routes them there. Left alone, *Revenue by Payment Method* would render a sixth, unlabelled bar holding a third of your bookings.

Fix it at the visual, once: add a visual-level filter `Payment Method → is not blank`, or equivalently `Status Group → is Successful or Failed`. Do not push a placeholder value into the fact table to make the blank go away.

Why every relationship is single-direction. Bi-directional filtering is the most common way a model like this goes wrong. It creates ambiguous filter paths as soon as a second fact table appears, it makes measures behave differently depending on which visual they sit in, and none of your five views needs a dimension to be filtered *by* the fact. If you later need a slicer to hide unused values, use `TREATAS` or a measure-level filter rather than flipping the arrow.

3.5 Building the relationships in Model View

1. Create the four DAX tables first (section 3.6) — **Home** → **New table**, once per table. Relationships cannot be drawn before both endpoints exist.
2. Switch to **Model view** (the third icon in the left rail).
3. Drag the tiles so `Fact_Rides` sits in the centre with the four dimensions around it. This is not cosmetic — a star that *looks* like a star is how you spot a wrong relationship at a glance.
4. Drag `Dim_Date[Date]` and drop it onto `Fact_Rides[Ride Date]`. Power BI creates the relationship and infers the cardinality.
5. Double-click the new connector to open **Edit relationship** and confirm three things: *Cardinality* is One to many (1:*) , *Cross-filter direction* is Single , and *Make this relationship active* is ticked. Leave *Assume referential integrity* unchecked — it only applies to DirectQuery.
6. Repeat steps 4–5 for the other three key pairs in the matrix above.
7. Confirm the arrows. Every one should point *from* the dimension *into* `Fact_Rides` . An arrowhead on the dimension end means you dragged in the wrong direction.
8. Delete any leftover auto relationship to `LocalDateTable_...` . If auto date/time was switched off in step 2.4 it should already be gone.
9. Hide the foreign-key columns in the fact from report view — right-click `Fact_Rides[Ride Date]` , `[Vehicle Type]` , `[Booking Status]` , `[Payment Method]` → *Hide in report view*. Report authors should reach these through the dimensions, so there is exactly one field named “Vehicle Type” in the field list.

3.6 DAX table definitions

DIM_DATE — HOME → NEW TABLE

```

Dim_Date =
VAR _Min = MIN ( Fact_Rides[Ride Date] )
VAR _Max = MAX ( Fact_Rides[Ride Date] )
VAR _Calendar =
    CALENDAR ( DATE ( YEAR ( _Min ), 1, 1 ), DATE ( YEAR ( _Max ), 12, 31 ) )
RETURN
ADDCOLUMNS (
    _Calendar,
    -- year and quarter
    "Year",          YEAR ( [Date] ),
    "Quarter Number", QUARTER ( [Date] ),
    "Quarter",       "Q" & QUARTER ( [Date] ),
    "Year Quarter",  YEAR ( [Date] ) & "-Q" & QUARTER ( [Date] ),
    -- month
    "Month Number",  MONTH ( [Date] ),
    "Month",         FORMAT ( [Date], "MMMM" ),
    "Month Short",   FORMAT ( [Date], "MMM" ),
    "Year Month",    FORMAT ( [Date], "yyyy-MM" ),
    "Month Year",    FORMAT ( [Date], "MMM yyyy" ),
    "Month Year Sort", YEAR ( [Date] ) * 100 + MONTH ( [Date] ),
    -- week (ISO-style, Monday start)
    "Week Number",   WEEKNUM ( [Date], 2 ),
    "Year Week",     YEAR ( [Date] ) & "-W" & FORMAT ( WEEKNUM ( [Date], 2 ),
    "Week Start Date", [Date] - WEEKDAY ( [Date], 2 ) + 1,
    -- day
    "Day",           DAY ( [Date] ),
    "Day of Week Number", WEEKDAY ( [Date], 2 ),
    "Day of Week",    FORMAT ( [Date], "dddd" ),
    "Day of Week Short", FORMAT ( [Date], "ddd" ),
    "Is Weekend",     WEEKDAY ( [Date], 2 ) ≥ 6,
    "Day Type",       IF ( WEEKDAY ( [Date], 2 ) ≥ 6, "Weekend", "Weekday" ),
    -- integer key, useful if you ever move to a warehouse source
    "Date Key",       YEAR ( [Date] ) * 10000 + MONTH ( [Date] ) * 100 + DAY (
)

```

Building the range from MIN / MAX of the fact and then widening to whole calendar years is deliberate: it guarantees complete years for year-over-year comparisons, and it keeps working when 2025 data lands without anyone editing the formula. For this extract it yields 366 rows, 2024-01-01 to 2024-12-31.

Sort-by-column settings — do these immediately

Without them every month axis reads April, August, December. Select the column, then **Column tools** → **Sort by column**:

SORT THIS COLUMN	BY THIS COLUMN
Month	Month Number
Month Short	Month Number
Month Year	Month Year Sort
Quarter	Quarter Number
Day of Week	Day of Week Number
Day of Week Short	Day of Week Number

Marking it as the official date table

- 1. Select `Dim_Date` in the Fields pane (Table view or Report view both work).
- 2. **Table tools** → **Mark as date table** → **Mark as date table**.
- 3. In the dialog, choose `Date` as the date column and click OK.

The column must be of type *Date*, contain no blanks and no duplicates, and cover a contiguous unbroken range — this table satisfies all three. Marking it does two things that matter: it tells the time-intelligence functions (`DATEADD` , `TOTALYTD` , `SAMEPERIODLASTYEAR`) which table to use as the calendar, and it removes the built-in date hierarchies that would otherwise shadow your own columns.

DIM_BOOKING_STATUS

```
Dim_Booking_Status =
DATATABLE (
    "Booking Status",          STRING,
    "Status Group",           STRING,
    "Is Revenue Generating",  BOOLEAN,
    "Status Sort",            INTEGER,
    {
        { "Completed",          "Successful",  TRUE,  1 },
        { "Incomplete",         "Failed",    TRUE,  2 },
        { "Cancelled by Customer", "Cancelled",  FALSE, 3 },
        { "Cancelled by Driver",  "Cancelled",  FALSE, 4 },
        { "No Driver Found",      "Unfulfilled", FALSE, 5 }
    }
)
```

Note the second row. *Incomplete* is marked revenue-generating because the audit proved it: all 9,000 incomplete rides carry a booking value, a distance

and a payment method. They are billed trips that ended badly, not lost bookings. Set *Sort by column* on Booking Status → Status Sort .

DIM_VEHICLE

```
Dim_Vehicle =
DATATABLE (
    "Vehicle Type",    STRING,
    "Vehicle Category", STRING,
    "Vehicle Sort",    INTEGER,
    {
        { "Bike",        "Two-Wheeler", 1 },
        { "eBike",       "Two-Wheeler", 2 },
        { "Auto",        "Auto",        3 },
        { "Go Mini",     "Hatchback",   4 },
        { "Go Sedan",    "Sedan",       5 },
        { "Premier Sedan", "Sedan",     6 },
        { "Uber XL",     "SUV / XL",    7 }
    }
)
```

DIM_PAYMENT_METHOD

```
Dim_Payment_Method =
DATATABLE (
    "Payment Method",  STRING,
    "Payment Category", STRING,
    "Payment Sort",    INTEGER,
    {
        { "UPI",        "Digital", 1 },
        { "Cash",       "Cash",    2 },
        { "Uber Wallet", "Digital", 3 },
        { "Credit Card", "Card",   4 },
        { "Debit Card",  "Card",   5 }
    }
)
```

Hard-coding the members rather than deriving them with

`DISTINCT(Fact_Rides[ ... ])` is the right call for a closed business domain: the dimension then defines what is *valid*, so a typo appearing in a future extract shows up as an unmatched blank member instead of silently becoming a new legitimate category.

3.7 Validation measures

Items 12–14 of the cleaning brief — validating ratings, booking value and ride